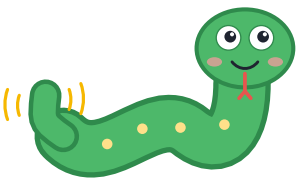
Lecture 6

Part 2: Decisions and Loops

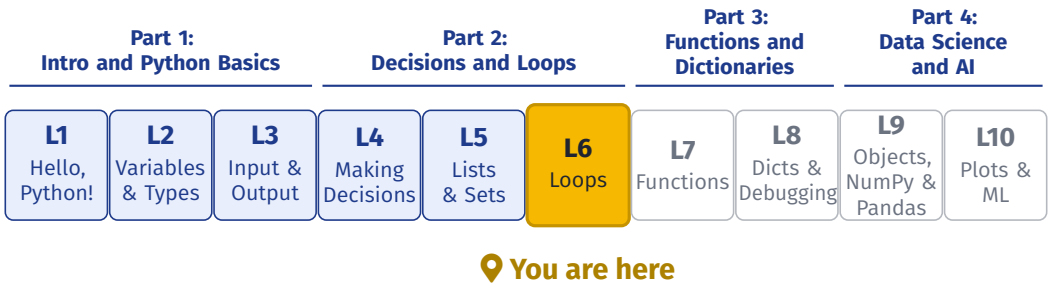
Loops

Teaching the intern to repeat: while, for, and loop recipes



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Wish 300 friends a happy new year?

Without a loop

```
print("Happy new year!")
print("Happy new year!")
print("Happy new year!")
```

...and 297 more lines. 😱

Wish 300 friends a happy new year?

Without a loop

```
print("Happy new year!")
print("Happy new year!")
print("Happy new year!")
```

...and 297 more lines. 😱

With a loop

```
for i in range(300):
    print("Happy new year!")
```

Two lines. Done.

Wish 300 friends a happy new year?

Without a loop

```
print("Happy new year!")
print("Happy new year!")
print("Happy new year!")
```

…and 297 more lines. 😱

With a loop

```
for i in range(300):
    print("Happy new year!")
```

Two lines. Done.

Today: make the intern repeat.



Today's plan

◎ By the end of today you can...

- Repeat code with `while` and `for`
- Walk through lists, strings and `range`
- Compute totals, averages, counts and maximums



You already loop every day



Playlist

on repeat



Snooze

until you get up



Bus 5

stops at every station



Pizza

a slice until it’s gone

You already loop every day



Playlist
on repeat



Snooze
until you get up



Bus 5
stops at every station



Pizza
a slice until it's gone

A loop: do it again, until a rule says stop.

Today's outline

1 **while loops**
Repeat as long as something is true; the loop that never ends

2 **for loops**
Repeat for each item, `range`, and `break`

3 **Loop recipes**
Total, average, count, max, search: five patterns for almost every loop

4 **The binary world, part 2 (extra)**
Bytes, letters as numbers, 13 in binary with a loop



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Computers shine at repetition: a million times, no mistakes
- Without loops you copy-paste. With loops, one line does it all
- The loop recipes solve most data questions

Where you will see it

- A step counter adding up your day
- Sending a message to every contact
- Photo filters: one rule for every pixel





Topic 1

while loops

Lecture 6 • Loops



TODAY

1. **while loops**
2. for loops
3. Loop recipes
4. Going further: the binary world, part 2

while: keep going as long as…

In real life

While there is pizza in the box, take a slice.

while: keep going as long as…

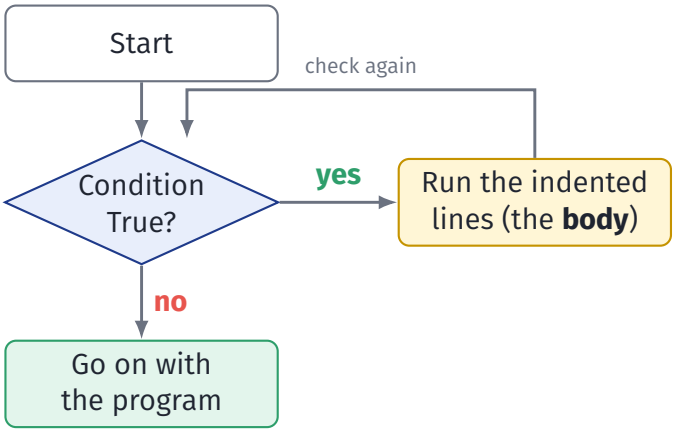
In real life

While there is pizza in the box, take a slice.



- Check a condition (True or False)
- True: do the steps, then check again
- False: stop and move on

How a while loop runs



Example: rocket countdown

```
count = 3
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```



The two indented lines are the **body**.
They repeat. The last line runs once.

Example: rocket countdown

```
count = 3
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```

OUTPUT

```
3
2
1
Liftoff!
```



The two indented lines are the **body**.
They repeat. The last line runs once.

Slow motion: what the intern does

count	`count > 0`?	prints
3	True	3

Slow motion: what the intern does

count	`count > 0`?	prints
3	True	3
2	True	2

Slow motion: what the intern does

count	`count > 0`?	prints
3	True	3
2	True	2
1	True	1

Slow motion: what the intern does

count	`count > 0?`	prints
3	True	3
2	True	2
1	True	1
0	**False**	Liftoff!



False means: leave the loop.

How many coffees can I afford?

```
money = 50
coffee_price = 12
cups = 0
while money >= coffee_price:
    money = money - coffee_price
    cups = cups + 1
print(f"I can buy {cups} coffees")
print(f"Money left: {money} ILS")
```



How many coffees can I afford?

```
money = 50
coffee_price = 12
cups = 0
while money >= coffee_price:
    money = money - coffee_price
    cups = cups + 1
print(f"I can buy {cups} coffees")
print(f"Money left: {money} ILS")
```



OUTPUT

```
I can buy 4 coffees
Money left: 2 ILS
```

We did not know in advance how many rounds. `while` found out.

Keep asking until it's right

```
password = input("Password? ")
while password != "falafel123":
    print("Wrong! Try again.")
    password = input("Password? ")
print("Welcome in!")
```



Keep asking until it's right

```
password = input("Password? ")
while password != "falafel123":
    print("Wrong! Try again.")
    password = input("Password? ")
print("Welcome in!")
```



OUTPUT

```
Password? pizza
Wrong! Try again.
Password? Password1
Wrong! Try again.
Password? falafel123
Welcome in!
```

The user typed three times.

Lecture 8: the same loop asks again until the user types a real number.

Careful: the loop that never ends

```
count = 3
while count > 0:
    print(count)
```



OUTPUT

```
3
3
3
3
```

...forever!

Careful: the loop that never ends

```
count = 3
while count > 0:
    print(count)
```

OUTPUT

```
3
3
3
3
```

…forever!



⚠ Infinite loop

`count` never changes, so `count > 0` stays True.

In Colab: click the cell's **stop** button.

Joke break: the never-ending shower

On the shampoo bottle:

Instructions

Lather. Rinse. **Repeat.**

Joke break: the never-ending shower

On the shampoo bottle:

Instructions

Lather. Rinse. **Repeat.**

The programmer is still in there.

Joke break: the never-ending shower

On the shampoo bottle:

Instructions

Lather. Rinse. **Repeat.**

The programmer is still in there.

The fix: say when to **stop**.
*"Repeat **while** your hair is still dirty."*



repeat, repeat, repeat…

Try it!

Try it in Colab notebooks/L06/L06_01_while_loops.ipynb

1. Count down from 10 to 1, then print `"Happy new year!"`
2. Change the coffee price. How many cups now?
3. Ask `"Are we there yet?"` until the user types `yes`




Predict the output

Predict the output!

```
n = 1
while n < 20:
    n = n * 2
print(n)
```



Predict the output

Predict the output!

```
n = 1
while n < 20:
    n = n * 2
print(n)
```

Answer

32

`n` goes 1, 2, 4, 8, 16, 32.
32 is not less than 20, so the loop stops.
`print` is not indented: it runs once.



Topic 2

for loops

Lecture 6 • Loops



TODAY

1. while loops
2. **for loops**
3. Loop recipes
4. Going further: the binary world, part 2

for: one round per item

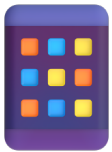
In real life

For each friend in the WhatsApp group, send an invite.

for: one round per item

In real life

For each friend in the WhatsApp group, send an invite.



- Walks through a list, item by item
- Stops by itself after the last item
- No counter to update, no infinite loop

Example: play the playlist

```
playlist = ["Shape of You", "Despacito", "Toxic"]
for song in playlist:
    print(f"Now playing: {song}")
print("Playlist over!")
```



Example: play the playlist

```
playlist = ["Shape of You", "Despacito", "Toxic"]
for song in playlist:
    print(f"Now playing: {song}")
print("Playlist over!")
```

OUTPUT

```
Now playing: Shape of You
Now playing: Despacito
Now playing: Toxic
Playlist over!
```
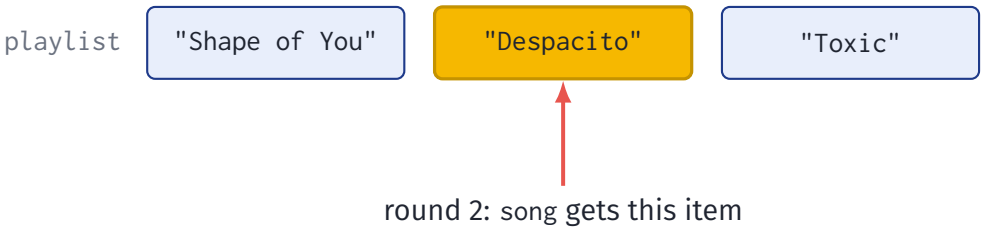


song is a new variable.
Python fills it for you.

Slow motion: who is song right now?



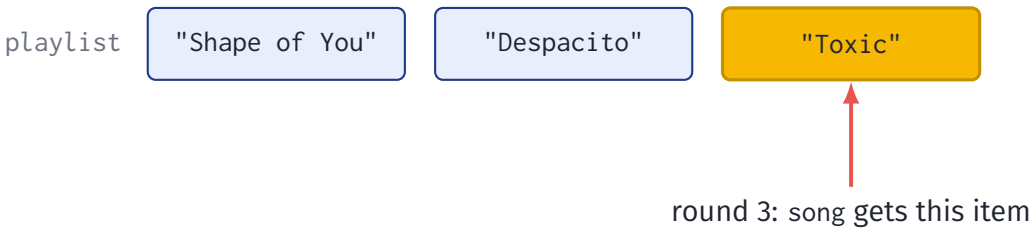


Slow motion: who is song right now?





Slow motion: who is `song` right now?





Slow motion: who is song right now?

playlist | "Shape of You" | "Despacito" | "Toxic"

No items left: the loop ends.



A string is a list of letters

```
for letter in "Pizza":
    print(letter)
```



A `for` loop walks through
lists **and** strings, item by item.

A string is a list of letters

```
for letter in "Pizza":
    print(letter)
```

A `for` loop walks through lists **and** strings, item by item.

OUTPUT

```
P
i
z
z
a
```



Repeat N times with `range`

```
for i in range(3):
    print("Hip hip hooray!")
```



Repeat N times with `range`

```
for i in range(3):
    print("Hip hip hooray!")
```

OUTPUT

```
Hip hip hooray!
Hip hip hooray!
Hip hip hooray!
```



Repeat N times with `range`

```python
for i in range(3):
    print("Hip hip hooray!")
```

OUTPUT

```
Hip hip hooray!
Hip hip hooray!
Hip hip hooray!
```



range(3)

Gives 0, 1, 2: three rounds.
You don't have to use `i`.

The many faces of `range`

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 20, 5)))
print(list(range(10, 0, -2)))
```

The many faces of `range`

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 20, 5)))
print(list(range(10, 0, -2)))
```

OUTPUT

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 5, 10, 15]
[10, 8, 6, 4, 2]
```

The many faces of `range`

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 20, 5)))
print(list(range(10, 0, -2)))
```

OUTPUT

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 5, 10, 15]
[10, 8, 6, 4, 2]
```

Same rule as slicing (Lecture 5)

`range(start, stop, step)`: start is in, stop is **never** in.

Example: the 7 times table

```
for i in range(1, 6):
    print(f"7 x {i} = {7 * i}")
```



This time we **use** `i`:
it is 1, then 2, …, then 5.

Example: the 7 times table

```
for i in range(1, 6):
    print(f"7 x {i} = {7 * i}")
```

This time we **use** `i`:
it is 1, then 2, …, then 5.

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
```



while or for?

Use `for` when...

- You have a list or a string
- You know how many rounds

Every song, every grade, 10 times

Use `while` when...

- You wait for something to happen
- You don't know how many rounds

Right password, money runs out



Emergency exit: `break`

```
playlist = ["Toxic", "Baby Shark", "Despacito"]
for song in playlist:
    if song == "Baby Shark":
        print("Nope. Stopping the music!")
        break
    print(f"Now playing: {song}")
```

Emergency exit: `break`

```
playlist = ["Toxic", "Baby Shark", "Despacito"]
for song in playlist:
    if song == "Baby Shark":
        print("Nope. Stopping the music!")
        break
    print(f"Now playing: {song}")
```

OUTPUT

```
Now playing: Toxic
Nope. Stopping the music!
```

`break` leaves the loop **right now**. “Despacito” never plays.

Try it!

Try it in Colab notebooks/L06/L06_02_for_loops.ipynb

1. Make a list of 3 friends. Print `"Hi <name>!"` for each
2. Print every letter of your name
3. Print the 9 times table, from 1 to 10



Predict the output

Predict the output!

```
for i in range(2, 10, 3):
    print(i)
```



Predict the output

Predict the output!

```
for i in range(2, 10, 3):
    print(i)
```



Answer

2
5
8

Start at 2, jump by 3. The next one, 11, is past 10.

Topic 3

Loop recipes

Lecture 6 • Loops



TODAY

1. while loops
2. for loops
3. **Loop recipes**
4. Going further: the binary world, part 2

The running total in your head

At the supermarket

You add each item to a total as it goes into the cart.





The running total in your head

At the supermarket

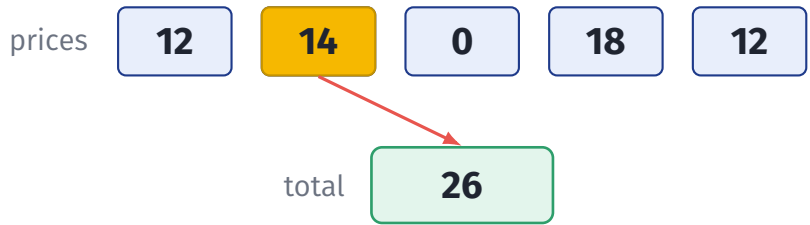
You add each item to a total as it goes into the cart.



The running total in your head

At the supermarket

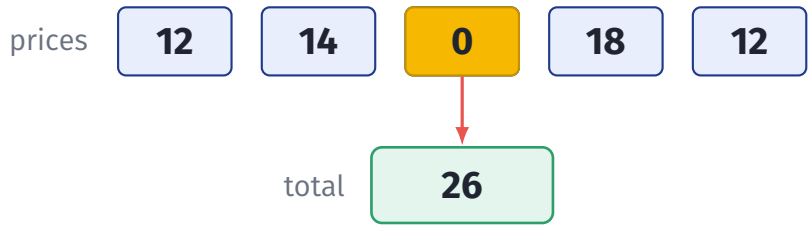
You add each item to a total as it goes into the cart.



The running total in your head

At the supermarket

You add each item to a total as it goes into the cart.



The running total in your head

At the supermarket

You add each item to a total as it goes into the cart.



The running total in your head

At the supermarket

You add each item to a total as it goes into the cart.



Start at 0, add each price. That's the **accumulator** pattern.

Recipe 1: the total

```
coffee = [12, 14, 0, 18, 12]  # ILS
total = 0
for price in coffee:
    total = total + price
print(f"Total: {total} ILS")
print(sum(coffee))
```



Recipe 1: the total

```python
coffee = [12, 14, 0, 18, 12]  # ILS
total = 0
for price in coffee:
    total = total + price
print(f"Total: {total} ILS")
print(sum(coffee))
```

OUTPUT

```
Total: 56 ILS
56
```



Recipe 1: the total

```
coffee = [12, 14, 0, 18, 12]  # ILS
total = 0
for price in coffee:
    total = total + price
print(f"Total: {total} ILS")
print(sum(coffee))
```

OUTPUT

```
Total: 56 ILS
56
```



`sum()` does the same.
Now you know what it does inside.

Recipe 2: the average

```
grades = [85, 92, 58, 74, 100, 66]
total = 0
for grade in grades:
    total = total + grade
average = total / len(grades)
print(f"Average: {average:.1f}")
```

Recipe 2: the average

```
grades = [85, 92, 58, 74, 100, 66]
total = 0
for grade in grades:
    total = total + grade
average = total / len(grades)
print(f"Average: {average:.1f}")
```

OUTPUT

```
Average: 79.2
```

Average = total ÷ how many.

475 points over 6 exams.

Recipe 3: counting

```
grades = [85, 92, 58, 74, 100, 66, 45]
passed = 0
for grade in grades:
    if grade >= 60:
        passed = passed + 1
print(f"{passed} of {len(grades)} passed")
```



Recipe 3: counting

```
grades = [85, 92, 58, 74, 100, 66, 45]
passed = 0
for grade in grades:
    if grade >= 60:
        passed = passed + 1
print(f"{passed} of {len(grades)} passed")
```

OUTPUT

```
5 of 7 passed
```



Recipe 3: counting

```
grades = [85, 92, 58, 74, 100, 66, 45]
passed = 0
for grade in grades:
    if grade >= 60:
        passed = passed + 1
print(f"{passed} of {len(grades)} passed")
```

OUTPUT

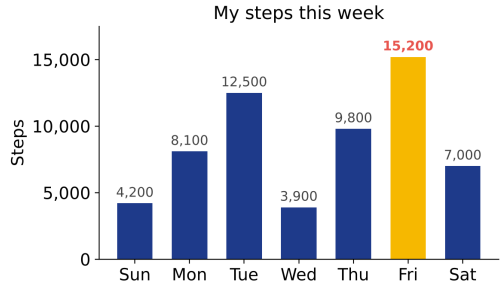
```
5 of 7 passed
```



A counter is an accumulator that adds **1**, only **if**…

Recipe 4: the maximum

```
steps = [4200, 8100, 12500, 3900,
         9800, 15200, 7000]
best = steps[0]
for day_steps in steps:
    if day_steps > best:
        best = day_steps
print(f"Best day: {best} steps")
print(max(steps))
```

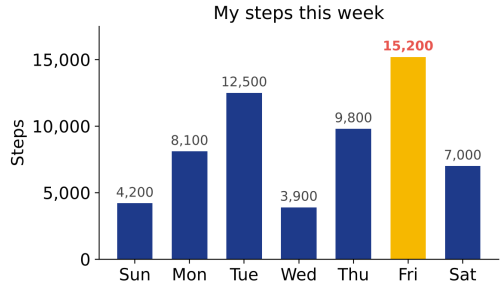


Recipe 4: the maximum

```
steps = [4200, 8100, 12500, 3900,
         9800, 15200, 7000]
best = steps[0]
for day_steps in steps:
    if day_steps > best:
        best = day_steps
print(f"Best day: {best} steps")
print(max(steps))
```

OUTPUT

```
Best day: 15200 steps
15200
```

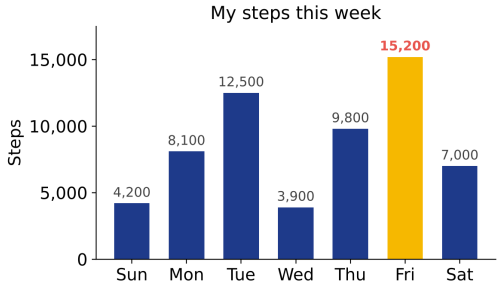


Recipe 4: the maximum

```
steps = [4200, 8100, 12500, 3900,
         9800, 15200, 7000]
best = steps[0]
for day_steps in steps:
    if day_steps > best:
        best = day_steps
print(f"Best day: {best} steps")
print(max(steps))
```

OUTPUT

```
Best day: 15200 steps
15200
```



The “tallest student” algorithm from Lecture 1!

Recipe 5: search

```
guests = ["Noa", "Omer", "Dana", "Yael"]
found = False
for guest in guests:
    if guest == "Dana":
        found = True
        break
print(f"Dana invited? {found}")
```



Recipe 5: search

```
guests = ["Noa", "Omer", "Dana", "Yael"]
found = False
for guest in guests:
    if guest == "Dana":
        found = True
        break
print(f"Dana invited? {found}")
```

OUTPUT

```
Dana invited? True
```



Recipe 5: search

```
guests = ["Noa", "Omer", "Dana", "Yael"]
found = False
for guest in guests:
    if guest == "Dana":
        found = True
        break
print(f"Dana invited? {found}")
```

OUTPUT

```
Dana invited? True
```



Shortcut from Lecture 5:
`"Dana" in guests`
does this loop for you.

All together: my coffee week

```
week = [12, 14, 0, 18, 12, 9, 16]
total = 0
big_days = 0
for price in week:
    total = total + price
    if price > 13:
        big_days = big_days + 1
print(f"Week total: {total} ILS")
print(f"Average: {total / len(week):.2f} ILS")
print(f"Days over 13 ILS: {big_days}")
```



All together: my coffee week

```
week = [12, 14, 0, 18, 12, 9, 16]
total = 0
big_days = 0
for price in week:
    total = total + price
    if price > 13:
        big_days = big_days + 1
print(f"Week total: {total} ILS")
print(f"Average: {total / len(week):.2f} ILS")
print(f"Days over 13 ILS: {big_days}")
```



OUTPUT

```
Week total: 81 ILS
Average: 11.57 ILS
Days over 13 ILS: 3
```

All together: my coffee week

```
week = [12, 14, 0, 18, 12, 9, 16]
total = 0
big_days = 0
for price in week:
    total = total + price
    if price > 13:
        big_days = big_days + 1
print(f"Week total: {total} ILS")
print(f"Average: {total / len(week):.2f} ILS")
print(f"Days over 13 ILS: {big_days}")
```



OUTPUT

```
Week total: 81 ILS
Average: 11.57 ILS
Days over 13 ILS: 3
```

One loop, two recipes at once.

Cheat sheet: four loop recipes

Recipe	Before the loop	Inside the loop
Total	`total = 0`	add the item
Count	`count = 0`	add 1, if…
Max	`best = first item`	bigger? remember it
Search	`found = False`	match? True, `break`



Set up before, update inside, use after.

Try it!

Try it in Colab notebooks/L06/L06_03_loop_recipes.ipynb

1. Your bus rides this week cost `[6, 6, 0, 12, 6]`. Total?
2. From a list of grades, count the ones above 90
3. Find the coldest day in a list of temperatures





Predict the output

Predict the output!

```
word = "banana"
count = 0
for letter in word:
    if letter == "a":
        count = count + 1
print(count)
```



Predict the output

Predict the output!

```
word = "banana"
count = 0
for letter in word:
    if letter == "a":
        count = count + 1
print(count)
```

Answer

3

b-**a**-n-**a**-n-**a**: the counter recipe on a string.



Bug Zoo: colon and indent

BUG ZOO

```
for i in range(3)
    print(i)
```

ERROR

```
SyntaxError: expected ':'
```

Fix: `for i in range(3):`

```
for i in range(3):
print(i)
```

ERROR

```
IndentationError: expected an
indented block after 'for'
statement on line 1
```

Fix: indent the body with 4 spaces



Same rules as **`if`** (Lecture 3): colon, then indent.

Bug Zoo: looping over a number

BUG ZOO

```
for i in 5:
    print(i)
```

ERROR

```
TypeError: 'int' object is not iterable
```

A number is not a list

“Iterable” = something you can walk through, item by item.

Fix: `for i in range(5):`



Bug Zoo: the silent bug

BUG ZOO

```
coffee = [12, 14, 0, 18, 12]
for price in coffee:
    total = 0
    total = total + price
print(total)
```

OUTPUT

```
12
```

We expected 56. No error message!

Reset inside the loop

`total = 0` runs in **every** round, so the total starts over each time.

Fix: move `total = 0` **above** the loop.

AI Corner: understand a loop

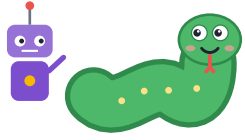
AI CORNER

A good prompt

"I'm learning Python loops. Here is my code: [paste]. Make a table: the value of each variable after each round."

Double-check

Run the code. Do the numbers in the AI's table match the real output?



Topic 4

Going further: the binary world, part 2

Lecture 6 • Loops



TODAY

1 while loops

2 for loops

3 Loop recipes

4 Going further: the binary world, part 2

The binary world, part 2

- You met 0s and 1s in **Lecture 4**
- Now you know **loops**, so let's peek a bit deeper
- For the curious: **no pressure**, it is a bonus

Look for the GOING FURTHER label. Just enjoy the ride.



Bits and bytes

GOING FURTHER

- A **bit**: one 0 or 1, like a light switch

```
print(2 ** 8)
print(2 ** 1, 2 ** 2, 2 ** 3)
```

Bits and bytes

GOING FURTHER

- A **bit**: one 0 or 1, like a light switch
- A **byte**: 8 bits in a row

```
print(2 ** 8)
print(2 ** 1, 2 ** 2, 2 ** 3)
```

Bits and bytes

GOING FURTHER

- A **bit**: one 0 or 1, like a light switch
- A **byte**: 8 bits in a row
- Each bit doubles the options:
  $2 \times 2 \times \cdots \times 2$

```
print(2 ** 8)
print(2 ** 1, 2 ** 2, 2 ** 3)
```

OUTPUT

```
256
2 4 8
```

Bits and bytes

GOING FURTHER

- A **bit**: one 0 or 1, like a light switch
- A **byte**: 8 bits in a row
- Each bit doubles the options: $2 \times 2 \times \cdots \times 2$

A byte can hold **256** different values: 0 to 255.

```
print(2 ** 8)
print(2 ** 1, 2 ** 2, 2 ** 3)
```

OUTPUT

```
256
2 4 8
```

Everything is numbers, even letters

GOING FURTHER

```
print(ord("A"))
print(chr(65))
for letter in "Hi":
    print(letter, ord(letter))
```

Everything is numbers, even letters

GOING FURTHER

```
print(ord("A"))
print(chr(65))
for letter in "Hi":
    print(letter, ord(letter))
```

OUTPUT

```
65
A
H 72
i 105
```

Everything is numbers, even letters

GOING FURTHER

```
print(ord("A"))
print(chr(65))
for letter in "Hi":
    print(letter, ord(letter))
```

OUTPUT

```
65
A
H 72
i 105
```

- `ord`: letter → its number
- `chr`: number → its letter

Everything is numbers, even letters

GOING FURTHER

```
print(ord("A"))
print(chr(65))
for letter in "Hi":
    print(letter, ord(letter))
```

OUTPUT

```
65
A
H 72
i 105
```

- `ord`: letter → its number
- `chr`: number → its letter
- Your WhatsApp message is a list of numbers, stored as bits

Everything is numbers, even letters

GOING FURTHER

```
print(ord("A"))
print(chr(65))
for letter in "Hi":
    print(letter, ord(letter))
```

OUTPUT

```
65
A
H 72
i 105
```

- `ord`: letter → its number
- `chr`: number → its letter
- Your WhatsApp message is a list of numbers, stored as bits 🟦
- A photo too: each pixel is numbers for its color (Lecture 9) 📷

13 in binary: keep dividing by 2

GOING FURTHER

number	remainder
13 // 2 = 6	**1**

- The remainder (`% 2`) is the next bit
- Stop when you reach 0

13 in binary: keep dividing by 2

GOING FURTHER

number	remainder
13 // 2 = 6	**1**
6 // 2 = 3	**0**

- The remainder (`% 2`) is the next bit
- Stop when you reach 0

13 in binary: keep dividing by 2

GOING FURTHER

number	remainder
13 // 2 = 6	**1**
6 // 2 = 3	**0**
3 // 2 = 1	**1**

- The remainder (`% 2`) is the next bit
- Stop when you reach 0

13 in binary: keep dividing by 2

GOING FURTHER

number	remainder
13 // 2 = 6	**1**
6 // 2 = 3	**0**
3 // 2 = 1	**1**
1 // 2 = 0	**1**

- The remainder (`% 2`) is the next bit
- Stop when you reach 0

13 in binary: keep dividing by 2

GOING FURTHER

number	remainder
13 // 2 = 6	**1**
6 // 2 = 3	**0**
3 // 2 = 1	**1**
1 // 2 = 0	**1**

- The remainder (`% 2`) is the next bit
- Stop when you reach 0
- Read the remainders from the **bottom up**:

1101

$8 + 4 + 0 + 1 = 13$ ✅

The same recipe as a while loop

GOING FURTHER

```
n = 13
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(bin(13))
```

The same recipe as a while loop

GOING FURTHER

```
n = 13
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(bin(13))
```

OUTPUT

```
1101
0b1101
```

The same recipe as a while loop

GOING FURTHER

```
n = 13
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(bin(13))
```

OUTPUT

```
1101
0b1101
```

- `n % 2`: the remainder, our next bit
- `str(...) + bits`: put it in **front**, so we read bottom up

The same recipe as a while loop

GOING FURTHER

```
n = 13
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(bin(13))
```

OUTPUT

```
1101
0b1101
```

- `n % 2`: the remainder, our next bit
- `str(...) + bits`: put it in **front**, so we read bottom up
- `n // 2`: divide and keep going

The same recipe as a while loop

GOING FURTHER

```
n = 13
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(bin(13))
```

OUTPUT

```
1101
0b1101
```

- `n % 2`: the remainder, our next bit
- `str(...) + bits`: put it in **front**, so we read bottom up
- `n // 2`: divide and keep going
- `bin(13)` agrees! (`0b` just means “binary”)

and, or, not with 0s and 1s

GOING FURTHER

a	b	a **and** b	a **or** b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	**not** a
0	1
1	0

```
print(True and False)
print(True or False)
print(not True)
print(int(True), int(False))
```

and, or, not with 0s and 1s

GOING FURTHER

a	b	a **and** b	a **or** b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	**not** a
0	1
1	0

```
print(True and False)
print(True or False)
print(not True)
print(int(True), int(False))
```

OUTPUT

```
False
True
False
1 0
```

and, or, not with 0s and 1s

GOING FURTHER

a	b	a **and** b	a **or** b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	**not** a
0	1
1	0

True is 1, **False** is 0: same tables, two spellings.

```
print(True and False)
print(True or False)
print(not True)
print(int(True), int(False))
```

OUTPUT

```
False
True
False
1 0
```

Predict the output: binary

GOING FURTHER

Predict the output!

```
n = 6
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(len("1" * 8), 2 ** 3)
print(chr(ord("A") + 2))
```

Predict the output: binary

GOING FURTHER

Predict the output!

```
n = 6
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
print(len("1" * 8), 2 ** 3)
print(chr(ord("A") + 2))
```

Answer

```
110
8 8
C
```

6: remainders 0, 1, 1, read bottom up. Two letters after A comes C.

Recap

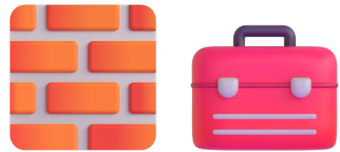
✔ Today you learned

- `while` repeats as long as a condition is True
- `for` visits every item of a list, string or `range`
- Loop recipes: total, average, count, max, search



Next time: write once, use forever

- Functions: give a recipe a name
- Send in values, get an answer back
- Build big programs from small bricks



3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md